



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Izveštaj iz predmeta
Projektovanje elektronskih
sistema

Sistem za automatsko zalivanje bašta

STUDENTI:

MARKO POPOVIĆ 19323

PETAR RISTIĆ 18881

PROFESOR:

PROF. DR BORISAV JOVANOVIĆ

Contents

1	Opis zadatka	2
2	Funkcionalna i nefunkcionalna specifikacija	3
3	Arhitektura sistema	4
4	Realizacija komunikacionog protokola	5
4.1	Komunikacija izmedju Master i Slejv automata	5
4.2	Komunikacija izmedju Master automata i Centralnog Računarskog sistema	6
5	Hardverska Realizacija automata	7
5.1	Prikaz sadržaja displeja Master automata	7
5.2	Hardverska realizacija Slejv automata	8
6	Realizacija firmvera Master automata	9
6.1	Detaljan opis najvažnijih funkcija sa segmentima koda	12
7	Realizacija firmvera Slejv automata	20
7.1	Definicija pinova i deklaracija promenljivih	20
7.2	Korišćene funkcije	21
7.3	Inicijalizacija	21
7.4	Obrada prekida	21
7.5	Glavna petlja i logika zalivanja	22
7.6	Nadzor protoka i alarm	22
7.7	LCD prikaz Slejv automata	22
8	Postupak verifikacije i vrednovanja koda	23
8.1	Rezultat rada sistema	23
8.2	Stepen ispunjenosti korisničkih zahteva	23
9	Detaljno uputstvo za upotrebu	24

1 Opis zadatka

Projektovati sistem za automatizovano zalivanje biljaka u baštama. Sistem sadrži sledeće funkcionalne celine: uređaje za zalivanje sa pumpom (do 16 Slejv uređaja) koji se instaliraju u baštama; jedan Master automat i centralni računarski sistem. Slejv uređaji su preko RS485 povezani sa Master automatom koji prikuplja informacije sa Slejv uređaja. Master automat je dalje povezan preko ETHERNET veze sa centralnim računarom.

Uređaj za zalivanje sa pumpom omogućava zalivanje biljaka po unapred definisanom programu rada. Zalivanje se odvija tako što u definisanom vremenskom trenutku početka zalivanja uređaj uključuje pumpu, a isključuje je posle određenog vremena. Vremena uključivanja pumpe i vremensko trajanje zalivanja definisani su programom rada. Postoji 16 različitih programa rada koji se zadaju sa centralnog računara. Pošto se smatra da je baštu dovoljno zalivati jednom dnevno, u svakom programu definisano je po jedno vreme uključivanja pumpe, vreme trajanja zalivanja (u sekundama). Uređaj za zalivanje poseduje taster za manuelno pokretanje/zaustavljanje procesa zalivanja. Kada se zalivanje pokrene manuelno, isključuje se ili manuelno ili automatski posle isteklog vremena trajanja zalivanja definisanog u tekućem programu rada. Manuelno zaustavljanje moguće je i kada zalivanje otpočne automatski. Protok vode kroz pumpu prati se očitavanjem senzora protoka. Ukoliko je protok izvan unapred zadatog opsega (minimalni i maksimalni protok) aktivira se zvučni alarm. Uređaj za zalivanje poseduje LCD na kome se prikazuju osnovni podaci o tekućem programu rada, statusu sistema, tekućem vremenu. Ukoliko je započeto zalivanje na LCD-u se prikazuje i koliko vremena je preostalo do isključivanja pumpe.

Centralni računar treba da omogućí:

- Očitavanje statusa uređaja za zalivanje (uključen/isključen, start/stop zalivanja, alarm...) svake sekunde.
- Definisanje svih 16 programa rada ili definisanje pojedinih vremenskih parametara (vreme uključivanja, trajanje zalivanja) za dati program rada
- Daljinsko podešavanje maksimalnog i minimalnog dozvoljenog protoka vode kroz pumpu
- Indikaciju alarma.
- Manuelno daljinsko uključivanje i isključivanje sistema, kao i daljinsko pokretanje i zaustavljanje procesa zalivanja

2 Funkcionalna i nefunkcionalna specifikacija

Funkcionalna specifikacija :

- Sistem se sastoji iz jednog Master i do 16 Slejv automata
- Master automat predstavlja centralni automat koji prikuplja informacije od svih Slejv automata i upravlja njima
- Master automatom se upravlja putem Centralnog Računarskog Sistema, a samim tim i Slejv automatima
- Slejv automati su kontroleri zalivanja koji se nalaze na svakoj bašti i upravljaju protokom i trajanju programa
- Master automat čuva statusne informacije od svih prisutnih Slejvova
- Master vrši kalibraciju Sata Realnog Vremena svih Slejv automata na zahtev CRS-a
- CRS ima funkciju prikazivanja statusa bašte, podešavanja Sata Realnog Vremena i programiranje programa rada
- Master automat čuva podatke o 16 programa zalivanja, koje dodeljuje Slejvovima na zahtev CRS-a
- Slejv automat odgovara periodično o svom trenutnom stanju
- Vrš se prikaz osnovnih podataka na Master ekranu

Nefunkcionalna specifikacija :

- Uredjaji moraju biti otporni na vlagu, vodu i prašinu, biti dobro izolovani, zbog svog rada u izloženim sredinama
- Takodje moraju imati malu potrošnju, i visoku pouzdanost

3 Arhitektura sistema

Arhitekturu sistema čine tri osnovne komponente sistema:

- Centralni Računarski Sistem (CRS)
- Master Automat
- Slave automat

CRS je sistem koji služi za upravljanje Master automatom i samim tim i Slejv automatima. Povezan je sa Masterom automatom putem Ethernet veze, statičkom IP adresom, na njihovoj lokalnoj mreži. Pristup Master automatu vrši se putem HTTP protokola, kucanjem statičke adrese direktno u browser.

Master automat upravlja Slejv automatima, kojih ima maksimalno 16. Vršiti se pozivka svakog Slejva na 125ms, pri kojoj se prikupljaju osnovni podaci o statusu svakog Slejva po protokolu koji će kasnije biti opisan. Automati su u slučaju kratkog udaljenosti međusobno povezani putem RS232 protokola, dok je u suprotnom korišćen RS485 protokol.

Slejv automat se nalazi kod pumpe svake bašte u sistemu, i upravlja protokom i vremenom operacije pumpe po prethodno definisanom programu koji mu je Master automat dodelio. Takođe ima funkciju alarma u slučaju prevelikog ili premalog protoka vode kroz sistem za zalivanje.

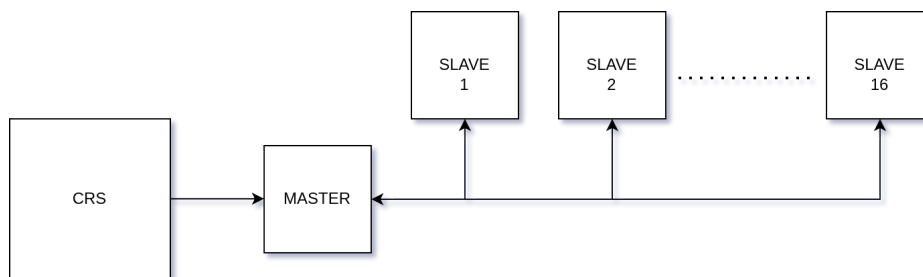


Figure 3.1: Dijagram arhitekture sistema

CRS ima jednosmernu komunikaciju ka Masteru, dok Master i Slejv automat realizuju obostranu.

4 Realizacija komunikacionog protokola

4.1 Komunikacija izmedju Master i Slevj automata

Komunikacija izmedju Master i Slevj automata realizovana je preko UART modula mikrokontrolera i RS485 drajvera. RS485 je izabran zato što omogućava rad više uređaja na istoj magistrali i pouzdan diferencijalni prenos na većim rastojanjima. U realizovanom sistemu komunikacija radi na bitskoj brzini od 19200 bps, a smer rada RS485 drajvera određuje signal DR. Kada automat šalje podatke, DR se postavlja na logičku jedinicu, a nakon slanja se vraća na nulu da bi automat ponovo slušao magistralu.

Master periodično proziva Slevj automate. Svaki Slevj ima jedinstveni identifikacioni broj od 0 do 15, koji se u firmveru Slevja čita sa donja četiri bita porta D:

$$\text{GARDEN_ID} = \text{PORTD} \& 0x0F$$

Na taj način jedna ista verzija programa može da se koristi na svim Slevj uređajima, a adresa uređaja se podešava prekidačima na razvojnoj ploči.

U implementiranom protokolu koriste se tri osnovna tipa poruka:

- **Prozivka statusa** – Master šalje jedan bajt oblika $0x20 \mid \text{ID}$. Slevj čiji se ID poklapa sa donja četiri bita poruke postavlja `CallFlag` i vraća odgovor od dva bajta.
- **Podešavanje sata realnog vremena** – Master šalje broadcast bajt $0x7F$, zatim tri bajta: sat, minut i sekund. Slevj primljene decimalne vrednosti konvertuje u BCD cifre koje koristi za interni sat.
- **Podešavanje programa zalivanja** – Master šalje bajt $0xA0 \mid \text{ID}$, zatim vreme početka programa i trajanje zalivanja. Slevj čuva primljene vrednosti i koristi ih pri automatskom startovanju pumpe.

Odgovor Slevj automata na prozivku ili uspešno podešavanje sastoji se iz komandnog bajta i statusnog bajta. Komandni bajt ima oblik $0x20 \mid \text{GARDEN_ID}$, čime Master može da proveri od kog Slevja je stigao odgovor. Statusni bajt koristi gornju polovinu bajta za najvažnija stanja uređaja:

Bit	Maska	Značenje
7	0x80	Sistem je uključen
6	0x40	Zalivanje je aktivno
5	0x20	Alarm protoka je aktivan
4	0x10	Zalivanje je pokrenuto manuelno

Preostala četiri bita statusnog bajta ostavljena su slobodna za eventualno proširenje protokola. Statusni bajt formira funkcija `buildStatusByte()`. Ona na osnovu internih promenljivih za uključen sistem, zalivanje, alarm i manuelni režim postavlja odgovarajuće maske.

Poruka za podešavanje programa zalivanja u realizovanom firmveru sadrži četiri korisna bajta:

Redni broj bajta	Promenljiva u Slejvu	Značenje
1	<code>Tmp_ProgStartHour</code>	Sat početka zalivanja
2	<code>Tmp_ProgStartMin</code>	Minut početka zalivanja
3	<code>Tmp_time_left_high</code>	Viši deo trajanja zalivanja
4	<code>Tmp_time_left_low</code>	Niži deo trajanja zalivanja

Kada se prijem ove poruke završi, postavlja se `ProgramSetupFlag`. Glavna petlja zatim prebacuje privremene vrednosti u aktivne promenljive programa. Vreme početka se konvertuje u BCD format, jer se u istom formatu vodi interni sat, dok se trajanje zalivanja računa kao:

$$\text{WateringSec} = \text{time_left_high} \cdot 100 + \text{time_left_low}$$

Time se omogućava da Master pošalje program rada, a Slejv da ga izvršava lokalno bez stalnog nadzora centralnog računara.

4.2 Komunikacija između Master automata i Centralnog Računarskog sistema

5 Hardverska Realizacija automata

5.1 Prikaz sadržaja displeja Master automata

Prikaz za Status

5.2 Hardverska realizacija Slejv automata

Slejev automat je realizovan na razvojnoj ploči sa mikrokontrolerom PIC16F877A. Mikrokontroler radi na taktu od 20 MHz i napajanju od 5 V, a u okviru sistema koristi digitalne ulaze i izlaze, UART komunikaciju, Tajmer 1, A/D konvertor i 16x2 LCD displej. Slejev je postavljen kod pumpe za zalivanje i zadužen je za lokalno upravljanje pumpom, očitavanje senzora protoka, prikaz stanja na displeju i komunikaciju sa Master automatom.

Najvažniji elementi Slejv automata su:

- mikrokontroler PIC16F877A,
- RS232/RS485 komunikacioni drajveri,
- 16x2 LCD displej,
- blok za napajanje razvojne ploče,
- konektor za programiranje mikrokontrolera,
- taster za manuelno pokretanje ili zaustavljanje zalivanja,
- taster za resetovanje lokalnog stanja,
- analogni ulaz za senzor protoka,
- LED indikatori za stanje sistema, zalivanje i alarm,
- prekidači za podešavanje ID broja Slejv automata.

Kao i u korišćenim razvojnim sistemima iz vežbi, izbor između RS232 i RS485 veze vrši se kratkospojnicima na ploči. RS232 je pogodan za razvoj i testiranje sa jednim uređajem na malom rastojanju, dok se u normalnom radu sistema koristi RS485, jer omogućava povezivanje većeg broja Slejv automata na zajedničku magistralu.

Raspored korišćenih pinova u realizovanom firmveru prikazan je u tabeli:

Pin/port	Smer	Namena
PORTB.F0	ulaz	Taster za manuelno pokretanje i zaustavljanje zalivanja
PORTB.F2	ulaz	Taster za resetovanje lokalnog stanja Slejva
PORTA.F0	ulaz	Analogni ulaz A/D konvertora za senzor protoka
PORTA.F2	izlaz	Indikacija da je sistem uključen
PORTA.F3	izlaz	Indikacija rada pumpe, odnosno aktivnog zalivanja
PORTA.F4	izlaz	Indikacija alarma protoka
PORTC.F5	izlaz	DR signal za izbor smera RS485 drajvera
PORTC.F6	izlaz	UART TX linija
PORTC.F7	ulaz	UART RX linija
PORTD.F0-F3	ulaz	Četvorobitni ID Slejv automata
PORTD.F4-F7	izlaz	Četvorobitna magistrala podataka LCD displeja
PORTC.F0	izlaz	RS kontrolni signal LCD displeja
PORTC.F2	izlaz	EN kontrolni signal LCD displeja

Na LCD displeju Slejv automata prikazuju se podaci potrebni za lokalni nadzor rada pumpe. Prvi red sadrži trenutno vreme i očitanu vrednost protoka, u obliku:

T HH:MM:SS Fxxx

Drugi red prikazuje da li su aktivni alarm i manuelni režim, vreme početka programa i preostalo vreme zalivanja:

AM S:HH:MM RMM:SS

Karakter A označava aktivan alarm, a karakter M manuelni režim. Kada odgovarajuće stanje nije aktivno, na tom mestu se prikazuje znak /.

6 Realizacija firmvera Master automata

Firmver Master automata pisan je u C jeziku i kompajliran na mikroC PRO kompajleru za PIC seriju mikrokontrolera. Kod za firmver se sastoji od funkcija za inicijalizaciju registara mikrokontrolera, inicijalizaciju globalnih promenljivih, funkcija za komunikaciju između automata i centralnog računarskog sistema, globalnog timer interrupta, funkcija za izpis na LCD i main funkcije.

U nastavku su date tabele makroa, pinova, varijabli, struktura i funkcija. Struktura

Figure 6.1: Tabela makroa i pinova

Naziv makroa ili pina	Dodeljena vrednost	Opis
SPI_Ethernet_HALFDUPLEX	0	Komunikacija sa CRS je u punom dupleksu
SPI_Ethernet_FULLDUPLEX	1	Komunikacija sa CRS je u punom dupleksu
DR	PORTA.F5	RS485 kontrola R/W
ButtonInc	PORTB.F0	Inkrement dugme za promenu prikaza
ButtonDec	PORTB.F1	Dekrement dugme za promenu prikaza

Mode opisuje program koji zadajemo putem browsera, sadrži sve potrebne elemente za programiranje pumpe bašte.

Figure 6.2: Struktura Mode

Naziv člana	Tip	Opis
startHour	unsigned char	Sat početka programa
startMinute	unsigned char	Minut početka programa
durationsH	unsigned char	Gornje dve cifre trajanja programa
durationsL	unsigned char	Donje dve cifre trajanja programa

Struktura Garden sadrži potrebni identifikacioni broj programa, kako bi smo osigurali da svaka bašta može da ima svih 16 postojećih programa.

Kako svaki član koristi isti tip podataka, izbegli smo padding, doprineli smo brzini programa i njegovoj čitljivosti i održavanju.

Figure 6.3: Struktura Garden

Naziv člana	Tip	Opis
modeID	unsigned char	Identifikacioni broj programa dodeljen bašti
gardenSend	unsigned char	Marker za slanje programa određenoj bašti

Figure 6.4: Tabela funkcija

Naziv funkcije	Tip	Opis
init()	void	Podešavanje rada pinova mikrokontrolera, konfigurisanje periferija kontrolera, A/D konvertera, prekida, kola tajmera i inicijalizacija UART-a
init_variables	void	Inicijalizacija početnih vrednosti promenljivih
interrupt()	void	Obrada prekida i debounce tastera
unsigned int SPI_Ethernet_UserTCP(unsigned char *remoteHost, unsigned int remotePort, unsigned int localPort, unsigned int reqLength, char *canCloseTCP)	unsigned int	Obraduju se zahtevi koji pristižu iz WEB pretraživača i izvršava se slanje podataka preko Etherneta
formBuffer()	void	Formira Niz podataka koji dobije od Slejva
appendBuffer()	void	Dodaje članove u niz
transmit()	void	Slanje podataka preko UARTA
updateLCD()	void	Ispis na LCD ekranu
lcdDisplayProgram(struct Mode Program, unsigned char ID)	void	Formatiranje prikaza programa
lcdDisplayUchar(unsigned char row, unsigned char col, unsigned char value)	void	Formatiranje prikaza unsigned chara
lcdDisplayBit(unsigned char mask)	void	Formatiranje prikaza bitova na ekranu
main()	void	Glavna funkcija programa u kojoj se vrši komunikacija i osvežavanje LCD-a

Figure 6.5: Tabela globalnih promenljivih

Naziv promenljive	Tip	Početna vrednost	Opis
Flag 1	unsigned char	0x00	Flag za 125ms
FlagRTC	unsigned char	0x00	Flag za slanje RTC po- dataka
FlagPoll	unsigned char	0x00	Flag da su svi Slejv atuo- mati prozvani
updateLCDFlag	unsigned char	0x00	Flag za osvežavanje LCD-a
SLAVE_ID	unsigned char	0x00	Jedinstveni identifikacioni broj svakog Slejv automata
ByteID	unsigned char	0x00	Jedinstveni identifikacioni broj svakog dolazećeg bajta
cmd[]	unsigned char	0x00	Komandni bajt svakog Sle- jva
Comm[]	unsigned char	0x00	Niz slejvova koji su se odaz- vali
Status[]	unsigned char	0x00	Status bajt koji svaki Slejv šalje Masteru na prozivku
no_ch	unsigned char	0x00	Brojač karaktera u nizu, može da se koristi i za pozi- cioniranje na početak niza
brojac	unsigned char	0x00	Brojač za tajmer 5x25ms
cntTas	unsigned char	0x00	Brojač za treperenje tastera
cntDisp	unsigned char	0x00	Brojač za prikaze na LCD-u
buffer[250]	unsigned char	0x00	Buffer za smeštanje odgov- ora od Slejvova

6.1 Detaljan opis najvažnijih funkcija sa segmentima koda

Funkcija `init_variables`

```
void init_variables() {  
  
    no_ch = 0x00;  
    ByteID = 0x00;  
    Flag1 = 0x00;  
    FlagRTC = 0x00;  
    FlagPoll = 0x00;  
    updateLCDFlag = 0x00;  
    btnCnt = 0x00;  
    cntDisp = 0x00;  
    hours = 0x00;  
    minutes = 0x00;  
    seconds = 0x00;  
    SLAVE_ID = 0x0F;  
    for (i = 0; i < 16; i++) {  
        Comm[i] = 0x00;  
        Status1[i] = 0x00;  
        Program[i].startHour = 0x00;  
        Program[i].startMin = 0x00;  
        Program[i].durationsH = 0x00;  
        Program[i].durationsL = 0x00;  
        Garden[i].modeID = 0x00;  
        Garden[i].gardenSend = 0x00;  
    }  
}
```

Funkcija `init`

```
void init() {  
  
    PIR1 = 0b00000000; // flegovi prijema preko UART-a  
    PIE1 = 0b00100001; // dozvola prekida za UART, RCIE, TMR1IE  
    // PIE1.TMR1IE = 1;  
    // PIE1.RC1IE=1;  
  
    T1CON = 0b10110000; // konfiguracija za tajmer1  
    T1CON.TMR1ON = 1;  
    /*16-bit operation  
    preskaler 1:8  
    25MH T0=40ns  
    40ns*4*8=1.28us  
    25ms=25000us=1.28*19531= B5B3 */  
    TMR1L = 0xB5;  
    TMR1H = 0xB3;  
  
    INTCON = 0b01000000; // periferijski interapt
```

```

INTCON.GIE = 1;          // globalna dozvola prekida

// svi pinovi na portu A su izlazi (vidi na semi)
TRISA = 0x00;
// obrati paznju, postoje tasteri na portovima RB0 - RB3, to su digitalni
// ulazi
TRISB = 0x0F; // ostali pinovi PORTB su izlazi
TRISC = 0xD0; // 0b11010000; // ovo je OK

PORTA = 0x00;
PORTB = 0x00;
PORTC = 0x00;

ADCON0 = 0x00; // isključujemo A/D konverziju
ADCON1 = 0x0F; // svi digitalni
// PORTA.F0=0;

UART1_Init(UART_BAUD_RATE);

// konfigurisemo brzinu od 19200 i dodatne bitove za serijski prenos
TXSTA.TXEN = 1;
RCSTA.SPEN = 1;
RCSTA.CREN = 1;
SPI1_Init_Advanced(_SPI_MASTER_OSC_DIV64, _SPI_DATA_SAMPLE_MIDDLE,
                  _SPI_CLK_IDLE_LOW, _SPI_LOW_2_HIGH);
SPI_Ethernet_Init(myMacAddr, myIpAddress, SPI_Ethernet_FULLDUPLEX);
Lcd_Init();
Lcd_Cmd(_LCD_CLEAR);
Lcd_Cmd(_LCD_CURSOR_OFF);
// UpdateLCD();
}

```

Funkcije appendBuffer i formBuffer

```

void appendBuffer(char *p_ch) {
    while ((*p_ch) != 0x00) {
        buffer[no_ch] = *p_ch;
        no_ch++;
        p_ch++;
    }
}

void formBuffer() {
    unsigned char i = 0;
    unsigned char txt[4];
    unsigned char StatusByte = 0x00;
    no_ch = 0x00; // pozicioniranje na pocetak niza
    for (i = 0; i < 16; i++) {
        if (Comm[i] == 1) { // samo slejuovi koji su se odazvali
            appendBuffer("Basta:");
            ByteToStr(i, txt);

```

```

appendBuffer(txt); // append broj baste
appendBuffer(" ");

StatusByte = Status1[i];
// prosto testiranje SWAM nibbla koji slave salje na
// master i ispisavnje statusa na master racunar

// B.J.: skрати tekst koji se ispisuje. Imas niz od samo 150 karaktera
// sve to moze krace
// za svaku bazu neka se ispisuju podaci u jednom redu
// tj. izbaci \n\n
// neka to samo bude na kraju, kada treba da se prikazuju podaci za
// narednu bazu

if (!(StatusByte & STATUS_SYSTEM_BIT)) {
    appendBuffer("OFF\n\n");
} else if (StatusByte & STATUS_WATER_BIT) {
    if (StatusByte & STATUS_MANUAL_BIT) {
        appendBuffer("Watering(Manual_Mode)\n\n");
        if (StatusByte & STATUS_ALARM_BIT) {
            appendBuffer("ALARM ON\n\n");
        }
    } else {
        appendBuffer("Watering(Automatic_Mode)\n\n");
        if (StatusByte & STATUS_ALARM_BIT) {
            appendBuffer("ALARM ON\n\n");
        }
    }
} else {
    appendBuffer("IDLE\n\n");
}
appendBuffer("<br>")
}
}
buffer[no_ch] = 0x00;
no_ch++; // kraj stringa
}

```

Funkcija transmit

```

void transmit(unsigned char DATA8b) {
    TXREG = DATA8b;
    while (!TXSTA.TRMT)
        ;
}

```

Funkcija interrupt

```

void interrupt() {
    if ((PIE1.TMR1IE == 1) && (PIR1.TMR1IF == 1)) {
        // prekid tajmera 1 - na svakih 25ms PIE1.TMR1IE = 1;
    }
}

```

```

PIR1.TMR1IF = 0;
if (brojac == 0x04) { // na svakih 125ms prozivka slejuova
    brojac = 0x00;
    Flag1 = 0x01;
} else {
    brojac++;
}
// 25ms

// B.J. neka bude ovako
if (btnCnt > 0)
    btnCnt--;

// B.J. ovo je greska
// B.J. stavlja zagrade svuda
// drugaciji je prioritet operacija

if ((ButtonInc == 1) && (btnCnt == 0))
// B.J. bilo: if ((ButtonInc == 1) && btnCnt == 0)
{
    btnCnt = 20; // 20*25=500ms
    updateLCDFlag = 1;
    if (cntDisp >= 20) {
        cntDisp = 0;
    } else {
        cntDisp++;
    }
}

// B.J. ovo je greska
if ((ButtonDec == 1) && (btnCnt == 0))
// if ((ButtonDec == 1) && btnCnt == 0)
{
    btnCnt = 20; // 20*25=500ms
    updateLCDFlag = 1;
    if (cntDisp == 0) {
        cntDisp = 20;
    } else {
        cntDisp--;
    }
}

// B.J.
// else if (btnCnt > 0)
//{
// btnCnt--;
//}

TMR1L = 0xB5;

```



```

    TMR1H = 0xB3; /// reset tajmera
}

if ((PIE1.RCIE) && (PIR1.RCIF)) { // UART RECIEVE FROM SLAVE
    unsigned char ch;
    PIR1.RCIF = 0; // Recieve flag na 0
    ch = RCREG; /// prima se bajt preko UART-a

    switch (ByteID) {
    case BYTE_ID_IDLE:
        break;
    case BYTE_ID_CMD_BYTE:
        if ((ch & CMD_TYPE_MASK) == STATUS_CODE) {
            if ((ch & CMD_ID_MASK) == SLAVE_ID) {
                Comm[SLAVE_ID] = 1;
                ByteID = BYTE_ID_STATUS_BYTE;
            } else {
                ByteID = BYTE_ID_IDLE;
            }
        } else {
            ByteID = BYTE_ID_IDLE;
        }
        break;
    case BYTE_ID_STATUS_BYTE:
        Status1[SLAVE_ID] = ch; // SWAM0000
        ByteID = BYTE_ID_IDLE;
        break;
    default:
        ByteID = BYTE_ID_IDLE;
        break;
    }
}
}

```

Funkcija lcdDisplayUchar

```

void lcdDisplayUchar(unsigned char row, unsigned char col,
                    unsigned char value) {
    unsigned char ones;
    unsigned char tens;
    tens = 0x00;
    ones = value;
    while (ones > 9) {
        ones = ones - 10;
        tens++;
    }
    Lcd_Chr(row, col, tens + '0');
    Lcd_Chr(row, col + 1, ones + '0');
}

```

Funkcija lcdDisplayBit

```

void lcdDisplayBit(unsigned char mask) {
    int i = 0;

    for (i = 0; i <= 15; i++) {
        if ((Status1[i] & mask) == mask) {
            Lcd_Chr(2, 16 - i, '1');
        } else {
            Lcd_Chr(2, 16 - i, '0');
        }
    }
}

```

Funckija lcdDisplayProgram

```

void lcdDisplayProgram(struct Mode Program, unsigned char ID) {

    // B.J. valjda su svi karakteri u oba reda definisani
    // proverite

    // Lcd_Cmd(_LCD_CLEAR);
    Lcd_Out(1, 1, "Prog:");
    lcdDisplayUchar(1, 6, ID);
    Lcd_Out(1, 9, " ");
    Lcd_Out(1, 13, "Totl"); // prvi red
    //=====drugi red=====
    lcdDisplayUchar(2, 1, Program.startHour);
    Lcd_Chr(2, 3, ':');
    lcdDisplayUchar(2, 4, Program.startMin);
    Lcd_Out(2, 6, " ");
    Lcd_Chr(2, 8, '/');
    lcdDisplayUchar(2, 9, Program.durationsH);
    lcdDisplayUchar(2, 11, Program.durationsL);
    Lcd_Out(2, 13, " ");
}

void updateLCD() {
    if (cntDisp <= 16) {
        lcdDisplayProgram(Program[cntDisp], cntDisp);
    } else if (cntDisp == 17) { // System
        Lcd_Out(1, 1, "Operation");
        lcdDisplayBit(STATUS_SYSTEM_BIT);
    } else if (cntDisp == 18) { // Watering
        Lcd_Out(1, 1, "Watering");
        lcdDisplayBit(STATUS_WATER_BIT);
    } else if (cntDisp == 19) { // Alarm
        Lcd_Out(1, 1, "Alarm");
        lcdDisplayBit(STATUS_ALARM_BIT);
    } else if (cntDisp == 20) { // Manual
        Lcd_Out(1, 1, "Manual");
        lcdDisplayBit(STATUS_MANUAL_BIT);
    }
}

```

```
}
```

Funkcija main

```
void main(void) {
    init();
    init_variables();

    while (1) {
        SPI_Ethernet_doPacket();
        if (Flag1 == 0x01) { // 125ms
            Flag1 = 0x00;
            SLAVE_ID++; // sledeca basta
            if (SLAVE_ID == 0x10) { // svi slejovi pollovan
                SLAVE_ID = 0x00;
                PORTA.F4 = 1;
            } else
                PORTA.F4 = 0;

            // B.J. organizacija ne valja,
            // saljes SAMO jedan paket, a ne sva tri odjednom

            if (Garden[SLAVE_ID].gardenSend == 0x01) {
                DR = 1;
                transmit(MODE_CODE | SLAVE_ID); // prvi komandni bajt
                ///// BrojBaste.BrojPrograma.Start 1Byte
                transmit(Program[Garden[SLAVE_ID].modeID].startHour);
                transmit(Program[Garden[SLAVE_ID].modeID].startMin);
                transmit(Program[Garden[SLAVE_ID].modeID].durationsH);
                transmit(Program[Garden[SLAVE_ID].modeID].durationsL);
                DR = 0;
                Garden[SLAVE_ID].gardenSend = 0x00;
                ByteID = BYTE_ID_CMD_BYTE;

                // Sanity check
                // B.J. Obavezno pisite u komentarima sta se dobija od slejva kao
                // odgovor koliko ima bajtova, kako izgledaju
            } else if (FlagRTC == 0x01) {
                // saljemo RTC signal svim bastama, da bi smo izbegli kasnjenja
                DR = 1;
                transmit(RTC_BROADCAST); // 0x1F
                // transmit(RTC_CODE|SLAVE_ID);
                transmit(hours);
                transmit(minutes);
                transmit(seconds);
                DR = 0;
                FlagRTC = 0x00;
                ByteID = BYTE_ID_CMD_BYTE;

                // Sanity check
            }
        }
    }
}
```

```

        // B.J. Obavezno pisite u komentarima sta se dobija od slejva kao
        // odgovor koliko ima bajtova, kako izgledaju
    } else {
        DR = 1;
        transmit(STATUS_CODE | SLAVE_ID); // pitamo za status slejva
        DR = 0;
        ByteID = BYTE_ID_CMD_BYTE; // ocekujemo cmd_byte

        // Sanity check
        // B.J. Obavezno pisite u komentarima sta se dobija od slejva kao
        // odgovor koliko ima bajtova, kako izgledaju
    }

} // od if (Flag1 == 0x01)

if (updateLCDFlag == 1) {
    updateLCDFlag = 0;
    updateLCD();
}
// od while(1)
}

```

7 Realizacija firmvera Slejv automata

Firmver Slejv automata napisan je za mikroC kompajler i kontroler PIC16F877A. Osnovna uloga firmvera je da lokalno upravlja procesom zalivanja i da Master automatu u svakom trenutku može da vrati ažuran status sistema. Zbog toga je program podeljen na inicijalizaciju periferija, obradu prekida, obradu korisničkih ulaza, komunikaciju i glavnu logiku zalivanja.

7.1 Definicija pinova i deklaracija promenljivih

Na početku programa definisane su simboličke oznake za ulaze i izlaze koji se koriste u kodu. Time se izbegava direktno korišćenje brojeva pinova u ostatku programa i olakšava se povezivanje opisa firmvera sa električnom šemom razvojne ploče.

Oznaka u kodu	Pin	Namena
PinTaster	PORTB.F0	Manuelno pokretanje ili zaustavljanje zalivanja
PinReset	PORTB.F2	Resetovanje lokalnog stanja automata
PinSystemOn	PORTA.F2	Izlazna indikacija da je sistem uključen
PinWatering	PORTA.F3	Izlazna indikacija rada pumpe
PinAlarm	PORTA.F4	Izlazna indikacija alarma protoka
DR	PORTC.F5	Upravljanje smerom RS485 dražvera

Promenljiva	Tip	Opis
GARDEN_ID	unsigned char	ID broj Slejv automata, očitava se sa PORTD.F0-F3
Hours, Minutes, Seconds	unsigned char	Trenutno vreme dobijeno dekodovanjem BCD cifara
ProgStartHour, ProgStartMin	unsigned char	Vreme početka automatskog zalivanja
WateringSec	unsigned int	Preostalo vreme zalivanja u sekundama
FlowValue	unsigned char	Trenutno očitana vrednost senzora protoka
FlowMin, FlowMax	unsigned char	Minimalna i maksimalna dozvoljena vrednost protoka
ManualMode	bit	Označava da je zalivanje pokrenuto manuelno
CallFlag	bit	Označava da je stigla prozivka od Master automata
RTCSetupFlag	bit	Označava da treba prepisati novo vreme u interni sat
ProgramSetupFlag	bit	Označava da treba prihvatiti novi program zalivanja
UpdateLCDFlag	bit	Označava periodično osvežavanje LCD displeja i logike zalivanja
m_bSystemOn, m_bWatering, m_bAlarm	bit	Interna stanja koja se pakuju u statusni bajt

7.2 Korišćene funkcije

Funkcija	Povratna vrednost	Opis
<code>init()</code>	<code>void</code>	Podešava smer portova, A/D konvertor, Tajmer 1, UART i prekide
<code>init_variables()</code>	<code>void</code>	Postavlja početne vrednosti globalnih promenljivih i čisti LCD
<code>interrupt()</code>	<code>void</code>	Obradjuje prekid Tajmera 1 i prijem bajtova preko UART-a
<code>ProcessInputs()</code>	<code>void</code>	Obradjuje tastere za manuelno zalivanje i reset sa debounce zaštitom
<code>IncrementTime()</code>	<code>void</code>	Uvećava interni sat za jednu sekundu
<code>DecodeTime()</code>	<code>void</code>	Pretvara BCD cifre sata u promenljive <code>Hours</code> , <code>Minutes</code> i <code>Seconds</code>
<code>ReadADC()</code>	<code>unsigned char</code>	Pokreće A/D konverziju i vraća vrednost protoka iz registra <code>ADRESH</code>
<code>buildStatusByte()</code>	<code>unsigned char</code>	Formira statusni bajt koji Slev šalje Masteru
<code>UpdateLCD()</code>	<code>void</code>	Ispisuje trenutno vreme, protok, alarme, program i preostalo vreme zalivanja
<code>transmit()</code>	<code>void</code>	Šalje jedan bajt preko UART predajnika

7.3 Inicijalizacija

Funkcija `init()` podešava hardver Slev automata. Port A se delimično koristi za analogni ulaz senzora protoka, a delimično za izlazne indikacije. Port B je podešen kao ulaz za tastere, port C sadrži UART linije, LCD kontrolne linije i DR signal RS485 drajvera, dok se port D koristi i za ID uredjaja i za podatke LCD displeja.

A/D konvertor je uključen preko registara `ADCON0` i `ADCON1`. Rezultat konverzije se koristi kao osmobarbna vrednost protoka, tako što se čita registar `ADRESH`. Tajmer 1 se podešava tako da prekid nastaje na svakih 100 ms. Početne vrednosti `TMR1H` = 0x0B i `TMR1L` = 0xDC odgovaraju intervalu od 100 ms za takt mikrokontrolera od 20 MHz i preskaler 1:8. UART se inicijalizuje na 19200 bps, a zatim se dozvoljavaju prekidi Tajmera 1 i prijema preko UART-a.

7.4 Obrada prekida

Funkcija `interrupt()` ima dve glavne grane. Prva obradjuje prekid Tajmera 1. Pošto tajmer generiše prekid na svakih 100 ms, brojač `Counter` se koristi da se posle deset intervala dobije period od jedne sekunde. Tada se pozivaju `IncrementTime()` i `DecodeTime()`, a `UpdateLCDFlag` se postavlja na jedinicu. U svakom tajmerskom prekidu poziva se i `ProcessInputs()`, čime se tasteri očitavaju dovoljno često da se pouzdano detektuje pritisak.

Druga grana prekida obradjuje UART prijem. Primljeni bajt se čita iz registra `RCREG`, a promenljiva `ByteID` odredjuje da li se očekuje prvi komandni bajt ili neki od bajtova korisnog sadržaja. Ako prvi bajt odgovara adresi trenutnog Sleva i tipu komande, postavlja se odgovarajući fleg:

- `CallFlag` za statusnu prozivku,

- `RTCSetupFlag` za podešavanje sata,
- `ProgramSetupFlag` za prihvatanje novog programa zalivanja.

Na ovaj način se u prekidu radi samo prijem i osnovno dekodovanje poruke, dok se duža obrada prebacuje u glavnu petlju.

7.5 Glavna petlja i logika zalivanja

Funkcija `main()` najpre poziva inicijalizacione funkcije za registre, promenljive i LCD, a zatim ulazi u beskonačnu petlju. U petlji se proveravaju flegovi koje su postavili prekidi i korisnički ulazi.

Reset događaj zaustavlja zalivanje, briše manuelni režim, gasi alarm i vraća sistem u osnovno uključeno stanje. Manuelni taster radi kao prekidač. Kada se manuelni režim uključi, pumpa se pokreće i u simulacionoj verziji programa dobija vreme rada od 180 sekundi. Ponovnim pritiskom tastera zalivanje se zaustavlja.

Automatsko zalivanje se pokreće kada su ispunjeni sledeći uslovi:

```
m_bWatering = 0,
Seconds = 0,
Hours = ProgStartHour,
Minutes = ProgStartMin.
```

Tada se preostalo vreme zalivanja postavlja na trajanje primljeno od Master automata, aktivira se izlaz pumpe i postavlja se status da je zalivanje u toku. Svake sekunde se `WateringSec` smanjuje za jedan. Kada dostigne nulu, izlaz pumpe se gasi i Slejv prelazi u stanje čekanja sledećeg programa ili manuelne komande.

7.6 Nadzor protoka i alarm

Vrednost senzora protoka očitava se funkcijom `ReadADC()`. Alarm se proverava samo dok je zalivanje aktivno. Ako je očitana vrednost manja od `FlowMin` ili veća od `FlowMax`, postavljaju se `PinAlarm` i `m_bAlarm`. U suprotnom se alarm gasi. U implementiranoj verziji početne granice su `FlowMin = 20` i `FlowMax = 60`. Time se detektuju i kvarovi sa premalim protokom, na primer zapušeno crevo ili prazna posuda, i stanja sa prevelikim protokom, na primer pucanje cevi.

7.7 LCD prikaz Slejv automata

Funkcija `UpdateLCD()` osvežava oba reda displeja. Prvi red prikazuje tekuće vreme i trenutnu vrednost protoka. Drugi red prikazuje alarm, manuelni režim, vreme početka programa i preostalo vreme do gašenja pumpe. Preostalo vreme se prikazuje u formatu minuta i sekundi pomoću funkcije `LcdOutDuration()`.

8 Postupak verifikacije i vrednovanja koda

8.1 Rezultat rada sistema

Verifikacija Slejv dela sistema vrši se povezivanjem Master i Slejv razvojne ploče preko RS485 veze i učitavanjem odgovarajućih firmvera u mikrokontrolere. ID Slejv automata podešava se prekidačima na donja četiri bita porta D. Nakon uključivanja napajanja proverava se da li se na LCD displeju prikazuje trenutno vreme, vrednost protoka, status alarma i preostalo vreme zalivanja.

Ispravnost komunikacije proverava se periodičnom prozivkom. Master na svakih 125 ms proziva sledeći Slejv i očekuje odgovor sa komandnim i statusnim bajtom. Kada se Slejv odazove, na strani Mastera se ažurira status odgovarajuće bašte, a preko CRS-a se može očitati da li je sistem uključen, da li je zalivanje aktivno, da li postoji alarm i da li je rad pokrenut manuelno.

Automatski rad proverava se slanjem programa zalivanja sa Master automata. Nakon prijema vremena početka i trajanja zalivanja, Slejv čuva program, prikazuje vreme početka na displeju i čeka poklapanje sa internim satom. Kada se vreme poklopi, uključuje se izlaz pumpe, pali se indikacija zalivanja, a preostalo vreme na displeju počinje da se smanjuje. Po isteku zadatog vremena pumpa se automatski isključuje.

Manuelni rad proverava se pritiskom tastera na PORTB.F0. Prvi pritisak pokreće zalivanje i postavlja status manuellnog režima, a drugi pritisak zaustavlja zalivanje. Reset taster na PORTB.F2 vraća Slejv u početno stanje i gasi alarm i pumpu.

Alarm protoka proverava se promenom analogne vrednosti na ulazu senzora protoka. Dok pumpa radi, vrednost ispod minimalne ili iznad maksimalne granice aktivira izlaz alarma i statusni bit alarma u odgovoru Slejva. Kada se vrednost vrati u dozvoljeni opseg, alarm se gasi.

8.2 Step en ispunjenosti korisničkih zahteva

Realizovani Slejv automat ispunjava osnovne zahteve lokalnog uredjaja za zalivanje. Uredjaj može da prihvati program rada od Master automata, da samostalno pokrene zalivanje u zadato vreme, da isključi pumpu po isteku trajanja programa i da omogući manuelno pokretanje ili zaustavljanje zalivanja. Interni sat može da se podesi daljinski preko Master automata, što omogućava uskladj en rad svih Slejv uredjaja u sistemu.

Status sistema dostupan je Master automatu kroz periodični odgovor Slejva. Statusni bajt sadrži informacije o uključenosti sistema, aktivnom zalivanju, alarmu i manuelnom režimu, što odgovara zahtevima za periodično očitavanje stanja bašte. Lokalni LCD prikazuje trenutno vreme, vrednost protoka, vreme početka programa i preostalo vreme zalivanja, pa korisnik na samom uredjaju može da prover i trenutno stanje.

Nadzor protoka je realizovan preko A/D konvertora i poređenja sa minimalnom i maksimalnom dozvoljenom vrednošću. Time je ispunjen zahtev za signalizaciju neispravnog protoka tokom rada pumpe. U trenutnoj verziji firmvera granice protoka su definisane u programu kao početne vrednosti, dok se daljinsko podešavanje granica može dodati proširenjem protokola na isti način kao što je realizovano podešavanje programa zalivanja.

9 Detaljno uputstvo za upotrebu

Pre korišćenja Slejv automata potrebno je podesiti njegov ID broj pomoću prekidača povezanih na `PORTD.F0-F3`. Svaki Slejv u sistemu mora da ima različit ID u opsegu od 0 do 15. Nakon toga se Slejv povezuje sa Master automatom preko RS485 konektora i uključuje se napajanje razvojne ploče.

Po uključanju, na LCD displeju Slejva treba da se prikažu trenutno vreme, vrednost protoka i status programa. Ako je potrebno sinhronizovati vreme, sa CRS-a se preko Master automata šalje komanda za podešavanje sata. Master zatim broadcast porukom šalje sat, minut i sekund svim Slejv uredjajima.

Program zalivanja se zadaje preko CRS-a izborom programa i dodelom tog programa odgovarajućoj bašti. Master tada šalje Slejvu vreme početka i trajanje zalivanja. Slejv prihvata program, upisuje ga u aktivne promenljive i na LCD displeju prikazuje zadato vreme početka. Kada interni sat dostigne zadato vreme, zalivanje se pokreće automatski.

Manuelno zalivanje pokreće se pritiskom tastera na `PORTB.F0`. Ako zalivanje nije bilo aktivno, pritisak tastera uključuje pumpu i postavlja manuelni režim. Ako je zalivanje već aktivno, sledeći pritisak tastera ga zaustavlja. Reset taster na `PORTB.F2` koristi se kada je potrebno odmah zaustaviti rad pumpe i vratiti lokalne statute u početno stanje.

Tokom zalivanja treba pratiti LCD displej i indikacione diode. Indikacija zalivanja pokazuje da je pumpa uključena, a alarmna indikacija se uključuje ako je protok izvan dozvoljenog opsega. U tom slučaju potrebno je proveriti dovod vode, pumpu, creva i senzor protoka. Kada se protok vrati u dozvoljene granice, alarm se automatski gasi pri sledećem očitavanju senzora.